ARI 2129 · Computer Vision for AI

Convolution Kernels

The Mathematical Engine of Modern Computer Vision

James Vella Gera, Gregory Vella, Nathan Tanti, Dusan Dinic, Dimitrios Paschalidis

What IS a Convolution Kernel?

The Problem

A computer sees an image as a massive grid of numbers.

127	89	210	45
200	132	78	255
90	33	145	201
88	176	230	55

The Flashlight Solution

Kernel = the flashlight
Stride = how far it moves
Feature Map = the map of what you found

Weight Sharing: Same filter, entire image → one 'flashlight' learns one pattern everywhere.

3×3 Kernel sliding over input:

0	0	0
0	1	0
0	0	0

Identity Filter

The Universal Dimension Equation

$$O = \lfloor (I - K + 2P) / S \rfloor + 1$$

I Input Size	**K** Kernel Size	**P** Padding	**S** Stride
Width or Height of input feature map	Width and height of the sliding filter. Odd numbers are normally used to have a centre pixel.	Zeros added around the input border before convolution	Determines how many pixels the kernel jumps between positions

Example:

```
Input=4, Kernel=3, Padding=0, Stride=1   →  O = (4−3+0)/1 + 1        = 2
Input=224, K=3, P=1, S=1                 →  O = (224-3+2)/1 + 1      = 224
```

Step-by-Step Calculation

Input (4×4)

1	1	1	0
1	1	1	0
1	1	1	0
0	0	0	0

Kernel (3×3)

0	0	0
0	1	0
0	0	0

Output (2×2)

1	1
1	1

Step 1: Multiply kernel over top-left 3×3 → 0·1+0·1+0·1+0·1+1·1+0·1+0·1+0·1+0·1 = **1**
Step 2: Slide the kernel to the right by 1 and repeat for the row below.
Verify: **O = (4–3+0)/1 +1 = 2**

The Specialist Families

Sobel — Edge Detection

−1	0	+1
−2	**0**	+2
−1	0	+1

Detects rapid brightness changes (edges). Computes image gradients in X and Y directions. Identifies outlines of shapes.

Gaussian — Smoothing

1	2	1
2	**4**	2
1	2	1

Blurs the image with a bell-curve weight. Removes noise and irrelevant details so the model focuses on structure.

Laplacian — Sharpening

0	−1	0
−1	**4**	−1
0	−1	0

Measures how rapidly an edge is changing. Detects 'edges of edges'. Used in medical imaging and texture analysis for fine details.

The Control Parameters

Stride (S)

$$O = \lfloor (I - K) / S \rfloor + 1$$

- S=1: kernel visits every pixel, maximum spatial detail preserved
- S=2: skips every other position, output halved, computation quartered
- Unlike pooling, strided convolution learns *what* to compress, not just *how*
- Large stride early in a network aggressively reduces memory cost

Padding (P)

$$\text{Same: } P = (K-1) / 2$$

- Without padding, a 3×3 kernel shrinks a 224px input by 2px per layer
- Corner pixels are seen once without padding, centre pixels up to K^2 times — padding rebalances this
- 'Valid' padding (P=0): output smaller, no artificial data added
- 'Same' padding (P=(K–1)/2): output matches input size exactly at S=1

Dilation (d)

$$K_{eff} = K + (K-1)(d-1)$$

- Inserts gaps between kernel weights
- Expands receptive field cheaply
- d=1 is standard convolution — no gaps
- d=2 on a 3×3 kernel covers the same area as a 5×5 kernel using only 9 weights

The Evolution of the Kernel

1980	**Neocognitron**	*Kernel hierarchy + shift invariance (Fukushima)*
1989	**TDNN**	*Kernels on 1D sequences — proved universality (Waibel)*
1998	**LeNet-5**	*Backprop + Conv — end-to-end training (LeCun)*
2012	**AlexNet**	*11×11 kernels, GPU, ReLU — deep learning era (Krizhevsky)*
2016	**Dilated Conv**	*Expanded receptive field at no extra cost (Yu & Koltun)*
2017	**MobileNets**	*Depthwise separable — 90% cheaper on mobile (Howard)*

Today: Dynamic Kernels + Attention Mechanisms (Transformers) are pushing the frontier further

Standard Convolution vs Depthwise Separable Convolution

Standard Convolution

One operation handles spatial patterns and channel mixing simultaneously

Cost: $K^2 \times C_{in} \times C_{out}$

Each of the 64 output channels requires a 3×3 filter applied across all 32 input channels simultaneously.
3 × 3 × 32 × 64 = **18,432 operations**

Depthwise Separable

Step 1 – Depthwise: 3×3 filter per channel independently.

Step 2 – Pointwise: 1×1 kernel mixes channels.

Cost: $K^2 \times C_{in} + C_{in} \times C_{out}$

Same example:
3 × 3 × 32 = **288 operations**
1 × 1 × 32 × 64 = **2,048 operations**
288 + 2,048 = **2,336 ops** *(~8× faster)*

Real-World Case Study

Kernels in Action: Self-Driving Cars, Medical AI & Mobile Vision

Tesla Autopilot

Dilated Convolutions

Dilated kernels (d=2,4,8) give the CNN a wide-angle view of the road without downsampling. The same 3×3 filter effectively sees a 15×15 area — crucial for detecting distant pedestrians while preserving lane-marking detail.

Medical Imaging

Sobel + U-Net

Radiologists use Sobel-seeded CNNs to segment tumour boundaries in MRI scans. The Laplacian kernel highlights micro-textures invisible to the human eye, enabling earlier cancer detection at sub-millimetre precision.

Google Photos (MobileNet)

Depthwise Separable

MobileNets use a smaller version of a standard CNN, but by using depthwise separable convolution — separating spatial filtering from channel mixing. The result: 8× fewer calculations and real-time classification of 1B+ photos daily without draining battery.

Failure Cases & Limitations

Fixed Receptive Field

Standard kernels have a fixed-size view. They can miss relationships between distant pixels unless dilation or many layers are used.

No Spatial Invariance Beyond Translation

Convolutions are translation-equivariant, NOT rotation or scale invariant. A '7' tilted 45° can fool the network unless augmented during training.

Boundary Artifacts (Zero Padding)

Zero-padding fills the image border with black pixels. Kernels near the edge multiply some weights against these zeros, producing weaker activations than identical patterns in the centre.

Checkerboard Artifacts (Strided Upsampling)

Transposed convolutions with stride > 1 create uneven pixel contributions, resulting in a checkerboard pattern in generated or segmented images. This occurs since some output pixels receive far more contributions than others.

Texture Bias

CNNs are biased toward texture rather than shape. A dog's fur pasted on a cat's shape is often classified as a dog.

High Computational Cost (Large K)

11×11 kernels (AlexNet) require 121× more operations than 1×1. Large kernels are expensive and rarely outperform stacked smaller ones.

When NOT to Use Convolution Kernels

✕ **Non-grid structured data**

Molecular graphs, social networks, irregular point clouds — no spatial locality to exploit

→ **Graph Neural Networks**

✕ **Long-range global dependencies dominate**

When context far away matters more than local patterns (e.g., document classification)

→ **Transformers**

✕ **Very small datasets (< few hundred samples)**

Kernels overfit badly without sufficient data

→ **Random Forest**

✕ **Tabular / time-series with irregular sampling**

Kernels assume uniform spacing; irregular timestamps break the assumption

→ **Recurrent Neural Network**

Key Papers

[2]

LeCun et al. (1998)

Gradient-Based Learning Applied to Document Recognition

LeNet-5: first end-to-end trained CNN. Blueprint of all commercial OCR.

[5]

Krizhevsky et al. (2012)

ImageNet Classification with Deep CNNs (AlexNet)

Kickstarted the deep learning era; introduced ReLU, dropout, GPU training.

[6]

Yu & Koltun (2016)

Multi-Scale Context Aggregation by Dilated Convolutions

Dilated kernels for dense prediction — foundational for segmentation and autonomous driving.

[7]

Howard et al. (2017)

MobileNets: Efficient CNNs for Mobile Vision Applications

Depthwise separable convolutions — enabled real-time AI on phones.

Key Takeaways

Kernels are like flashlights scanning for patterns using weight sharing

$O = \lfloor (I - K + 2P)/S \rfloor + 1$

Sobel/Gaussian/Laplacian are the base of all learned filters

Depthwise separable convolution leads to around 8× fewer operations

Fails on rotation, scale, non-grid data, long-range context

"The kernel is the flashlight of the mind"